HCM LITE — ENTERPRISE HRMS

Comprehensive System Architecture, AI Biometrics & Judge Defense Guide

Built for Saylani Hackathon Defense | Authoritative Technical Reference

1. EXECUTIVE SUMMARY & TECH STACK OVERVIEW

HCM Lite is a high-performance, multi-tenant Human Capital Management system. It features AI-powered biometric facial verification, GPS geofencing with the Haversine trigonometric formula, multi-tenant chunking, automated FBR-compliant payroll processing, and an intelligent GenAI Assistant with tool calling.

Frontend Framework: React 19 (Single Page Architecture, Functional Hooks, Vite Bundler, Tailwind CSS v4)
Backend Runtime: Node.js LTS with Express.js server, TypeScript, RESTful JSON APIs
Database Engine: SQLite via WebAssembly (sql.js) with persistent binary synchronization to container storage
Biometrics Engine: TensorFlow.js (@vladmandic/face-api) - SSD MobileNet V1 + ResNet-34 128-d Feature Vector Extractor
AI Intelligence SDK: @google/genai (Gemini 3.5 Flash) with server-side Multi-Tool Function Calling
Security & Auth: Stateless JWT (HMAC-SHA256, 7-day expiration), bcryptjs password hashing (salt rounds = 10)

2. DATABASE DEPLOYMENT & STORAGE ARCHITECTURE

Database Name: SQLite 3 (Embedded Relational SQL Engine compiled via WebAssembly - sql.js)
Storage Location: /app/applet/data/hcm.sqlite (Binary file on the local filesystem / persistent container volume)
Deployment Platform: Google Cloud Run / Linux Container (Port 3000 entry point, auto-scaling, persistent volume)
Persistence Logic: On boot, binary file is loaded into WASM memory. Every INSERT/UPDATE/DELETE triggers an atomic binary write (fs.writeFileSync) to disk, ensuring zero data loss upon container cold restarts.
Multi-Tenancy Chunking: All relational tables (departments, employees, attendance, leave_requests, payroll_runs, payslips, public_holidays, audit_logs) enforce strict tenant isolation via company_id indexed foreign keys. Each company sees exclusively their own partitioned data chunk.

3. BIOMETRIC AI FACE VERIFICATION — DEEP TECHNICAL SPECS

The biometric attendance verification system guarantees that proxy attendance and buddy punching are impossible. Raw camera frames are NEVER stored; faces are transformed into 128-dimensional mathematical feature vectors.

A. Face Detection Model

Architecture: SSD MobileNet V1 (Single Shot MultiBox Detector with MobileNet backbone)

Training Dataset: WIDER FACE Benchmark Dataset (32,203 high-resolution images containing 393,703 labeled faces with severe variations in pose, occlusion, expression, scale, and lighting conditions)

Function: Locates the face bounding box in real-time camera feed at 30 FPS with confidence scoring (threshold >= 0.50).

B. Facial Feature Descriptor & Recognition Model

Architecture: ResNet-34 Deep Convolutional Neural Network (adapted for 128-d facial landmark embeddings)

Training Dataset: Labeled Faces in the Wild (LFW) dataset + VGGFace2 dataset (3.31 million face images across 9,131 distinct identities)

Output Representation: 128-dimensional floating point vector (embedding descriptor): [x0, x1, x2, ..., x127]

Mathematical Matching: Euclidean Distance metric between check-in vector (V_in) and check-out vector (V_out):

```
Distance (d) = sqrt( sum( (V_in[i] - V_out[i])^2 ) for i = 0 to 127 )
```

Matching Rule: If d <= 0.55 -> MATCH VERIFIED (Same Employee). If d > 0.55 -> REJECTED (Different Person / Impostor).

C. Privacy, Anti-Spoofing & Geofencing Security

Privacy Compliance: GDPR/Data Protection Compliant. No biometric JPEG/PNG images exist on server disks. Vectors are mathematically non-invertible (cannot reconstruct original face photos from 128 floats).

GPS Haversine Formula: Server calculates spherical distance d = 2 * R * asin(sqrt(sin^2(dLat/2) + cos(lat1)*cos(lat2)*sin^2(dLng/2))) where R = 6,371,000m. Rejects check-in if distance > 150m.

Server Timestamp Authority: Check-in and check-out times are set strictly by the backend server clock (UTC/PKT). Device/client clock manipulation is completely neutralized.

Duplicate Idempotency: Relational UNIQUE constraint on (employee_id, date) returns HTTP 409 Conflict if an employee tries multiple check-ins on the same calendar day.

4. GEMINI AI ASSISTANT & MULTI-TOOL FUNCTION CALLING

The HR/Admin AI Assistant uses the official Google GenAI SDK (@google/genai) connected to Gemini 3.5 Flash. Rather than generating hallucinations, it queries live database records using structured tool calling.

AI Model: Gemini 3.5 Flash with automatic fallback chain: [gemini-3.5-flash, gemini-3.7-flash, gemini-3.6-flash]

SDK Used: @google/genai TypeScript SDK using ai.chats.create with multi-turn conversation session

Registered Tools: 1. get_headcount 2. search_employees 3. get_daily_attendance_summary 4. get_leave_balances 5. get_payroll_summary 6. list_departments

Tenant Security Boundary: The authenticated user's company_id is bound server-side to every tool execution handler. The LLM can NEVER access or leak data belonging to other companies, even if prompted maliciously.

5. JUDGE Q&A CHEATSHEET — 10 VITAL TECHNICAL DEFENSE QUESTIONS

Q: Which database are you using, and where is the data physically stored?

Roman Urdu: Aapka database konsa hai aur data kahan save hota hai?

Ans: We use SQLite 3 running via WebAssembly (sql.js) inside Node.js. The binary database file is physically persisted at /app/applet/data/hcm.sqlite on the application container's storage volume (deployed on Google Cloud Run). After every write operation, fs.writeFileSync commits the changes to disk, ensuring ACID compliance and persistence across container restarts.

Q: How do you isolate data between multiple companies (multi-tenancy)?

Roman Urdu: Har account ya company ka data alag alag kaisay rehta hai?

Ans: We implement multi-tenant chunking via schema-level foreign key isolation. Every table (employees, attendance, payroll, leaves, departments) has an indexed company_id column. When a user authenticates, their company_id is securely extracted from their JWT token server-side. All SQL queries automatically append "WHERE company_id = ?" ensuring complete privacy and isolation.

Q: What AI model powers facial verification, and what dataset was it trained on?

Roman Urdu: Face recognition kay liye konsa model use kiya aur kis dataset pr train hua hai?

Ans: We use a 2-stage deep learning pipeline from TensorFlow.js (@vladmandic/face-api): (1) Face Detection uses SSD MobileNet V1 trained on the WIDER FACE benchmark dataset (32k+ images, 393k+ faces). (2) Face Recognition uses a ResNet-34 deep convolutional network trained on LFW (Labeled Faces in the Wild) and VGGFace2 (3.3 million images across 9,000+ identities) to generate a 128-dimensional embedding vector.

5. JUDGE Q&A CHEATSHEET (CONTINUED)

Q: How does the system compare check-in and check-out faces?
Roman Urdu: Check-in aur check-out kay waqt face match kaisay hota hai?
Ans: During check-in, the ResNet-34 network extracts a 128-dimensional vector of floating-point numbers representing the user's facial features and saves it to attendance.face_descriptor. During check-out, a new 128-d vector is generated from the webcam. The system computes the Euclidean distance between the two vectors. If the distance is <= 0.55, the checkout is approved as verified; if > 0.55, checkout is blocked with an identity mismatch error.

Q: Are user face images stored on your database or server?
Roman Urdu: Kya user ki shakal ya camera photos database mein save hoti hain?
Ans: No! Raw camera images or photos are NEVER stored anywhere on disk or in the database. Only the 128-dimensional mathematical floating-point embedding vector is stored. This vector is irreversible, meaning an attacker or unauthorized party cannot reconstruct the human face from it, fulfilling stringent biometric privacy regulations.

Q: How do you prevent employees from spoofing GPS location or changing device time?
Roman Urdu: Employee mobile time ya GPS fake karke check-in kaisay rok sakte hain?
Ans: Two server-authoritative controls: (1) Time Authority: The check-in and check-out timestamps are generated strictly by the server system clock (new Date().toISOString()), completely ignoring client device time. (2) GPS Validation: The server calculates the exact distance using the Haversine spherical formula against the company office coordinates and enforces an accuracy cutoff (<= 100m) to reject GPS spoofing and coarse cellular approximations.

Q: How is Pakistani tax (FBR) calculated in payroll?
Roman Urdu: Payroll mein tax calculation kaisay hoti hai?
Ans: We implement the exact FBR income tax slabs for salaried individuals: (1) Up to 600k PKR/year = 0%. (2) 600k to 1.2M PKR/year = 5% of amount exceeding 600k. (3) 1.2M to 2.2M PKR/year = 30k + 15% exceeding 1.2M. (4) 2.2M to 3.2M = 180k + 25% exceeding 2.2M. (5) 3.2M to 4.1M = 430k + 30% exceeding 3.2M. (6) Above 4.1M = 700k + 35%. The annual tax is divided by 12 for monthly withholding.

Q: How does the AI Assistant execute actions without hallucinating?
Roman Urdu: AI assistant database ka sahi data kaisay lata hai bina ghalti ke?
Ans: We use Gemini 3.5 Flash with Function Calling (Tools). When the user asks "How many employees are late today?", the model does not invent numbers. It returns a structured tool call get_daily_attendance_summary(). Our Node.js backend executes the query scoped to the user's company_id, returns the true JSON data to the model, and Gemini formats it into a human-friendly response.

Q: What happens if the company has no departments or leave types yet?
Roman Urdu: Agar company nayi bani ho aur koi department na ho to crash hota hai?
Ans: The application has zero-dummy-data architecture. It starts completely clean. If an admin navigates to employees or payroll before creating departments, guided empty states with direct call-to-actions ("No departments yet — create your first department") appear gracefully without any application crashes.

Q: Why did you choose SQLite over an external PostgreSQL database for this build?
Roman Urdu: Aap nay SQLite kyu use kiya external SQL database ke bajaye?
Ans: SQLite via sql.js provides zero-latency in-process execution, zero cloud dependency failures during hackathon judging, full ACID compliance, and zero cost while preserving standard relational SQL query syntax. For high-scale production, this schema seamlessly migrates to PostgreSQL or Cloud SQL because all table relations, constraints, and queries follow ANSI SQL standards.